

What is a binary file?

In contrast to *text files* Purely, it stores data as characters arranged in zero or more lines. *binary files* Storing data more efficiently as a string of bytes, which is generally understood to be something other than text characters, is a common practice. Executable files are often referred to as... *binary file* In fact, binary files can contain any type of content.

Text Files vs Binary Files

- text files contain "text", usually in ASCII
- everything else is a binary file
- 1234 vs "1234"

byte 0	byte 1	byte 2	byte 3
00000000	00000000	00000100	11010010
0	0	4	210
49	50	51	52

Some binary files contain *metadata* It's important to provide information about how to interpret the data in a file. For example, a file signature, also known as a

digital signature.*magic number*Executable files can identify file formats, much like labels on products in a grocery store. Executable files contain binary code that causes the computer to perform specific tasks as instructed in the code.

Let's consider a plain text file, `bugprove.txt` contains a notification "*Hello from BugProve*" and the binary content of a binary file can be executed `bugprove`. The same message is displayed in the console. The following screenshot shows the raw contents of the files in hexadecimal format using a hexdump utility called `xxd`. The content of the plain text file is printed out by **cat command**, one of the most frequently used commands in Linux, and the program's console output.

The first five bytes `48 65 6c 6c 6c 6c` represent characters `Helloin ASCII`, a standard data encoding format that assigns numerical values to characters. Note that the contents of the executable binary file, shown on the right side of the screenshot above, are not in a format easily readable by humans, even though the program prints the same message. Furthermore, you may also notice a special identifier (magic number). `7F 45 4c 46` In the first four bytes, it indicates that this is a file. `ELF` binary. However, in the case of JPG image files, you will see `FF D8`. See **file signature list**. Continuously updated by Gary Kessler for more examples.

ASCII	Char	Hex	Bin		ASCII	Char	Hex	Bin
65	A	41	0100 0001		97	a	61	0110 0001
66	B	42	0100 0010		98	b	62	0110 0010
67	C	43	0100 0011		99	c	63	0110 0011
68	D	44	0100 0100		100	d	64	0110 0100
69	E	45	0100 0101		101	e	65	0110 0101
70	F	46	0100 0110		102	f	66	0110 0110
71	G	47	0100 0111		103	g	67	0110 0111
72	H	48	0100 1000		104	h	68	0110 1000
73	I	49	0100 1001		105	i	69	0110 1001
74	J	4A	0100 1010		106	j	6A	0110 1010
75	K	4B	0100 1011		107	k	6B	0110 1011
76	L	4C	0100 1100		108	l	6C	0110 1100
77	M	4D	0100 1101		109	m	6D	0110 1101
78	N	4E	0100 1110		110	n	6E	0110 1110
79	O	4F	0100 1111		111	o	6F	0110 1111
80	P	50	0101 0000		112	p	70	0111 0000
81	Q	51	0101 0001		113	q	71	0111 0001
82	R	52	0101 0010		114	r	72	0111 0010
83	S	53	0101 0011		115	s	73	0111 0011
84	T	54	0101 0100		116	t	74	0111 0100
85	U	55	0101 0101		117	u	75	0111 0101
86	V	56	0101 0110		118	v	76	0111 0110
87	W	57	0101 0111		119	w	77	0111 0111
88	X	58	0101 1000		120	x	78	0111 1000
89	Y	59	0101 1001		121	y	79	0111 1001
90	Z	5A	0101 1010		122	z	7A	0111 1010

What is the binary file format?

Overall, *file format* These are data structures that contain the information necessary for other programs to know how to handle the file, including the operating system's loader to manage the executable code packaged in binary files.

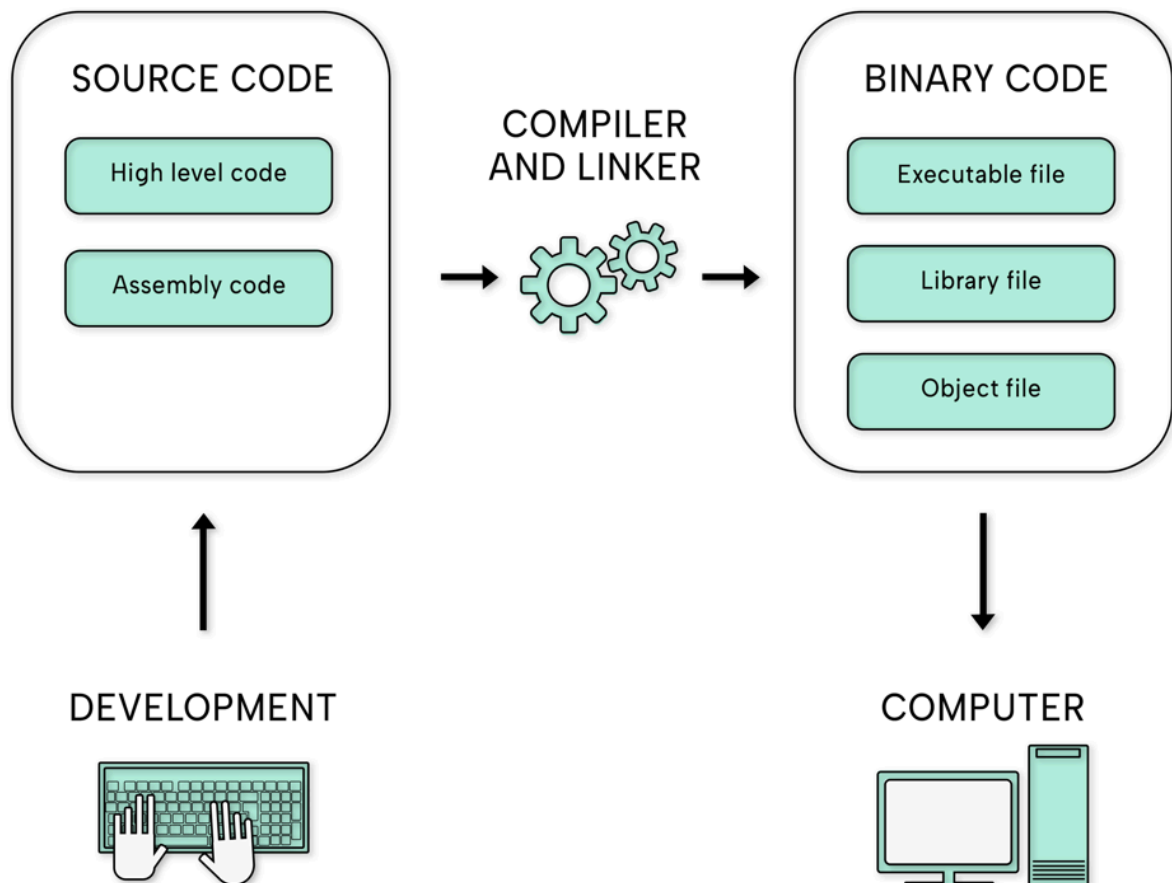
Developers create executable binary files via *compilation process* and *link* by converting human-written source code, such as C or C++, into machine code for the computer to execute. *Compiler* It only generates machine language commands from source code and creates one or more object files that you cannot yet run. *The link creates an executable binary file from the object files, which the loader* The operating system's data can be placed in memory and prepared for execution.

Executable files begin with a header that includes information about the source code, application type, required library functions, and size requirements.

Format ELF (*Executable and Linkable Format*) is a common standard binary file format for executable files, object code, shared libraries, and error dump files on Linux systems, while the file format **ON** (*Portable Executable*) is used by executable files, object code, and DLLs on Windows systems. Most binary formats have many similarities and a few essential differences. One obvious difference is the file extension: for example, the filenames of PE binary files end in `.py.exe`, whereas ELF binary files do not have an extension.

What is binary analysis?

Binary analysis It is the process of examining the properties of binary files, including the commands and data encoded in binary form within them, to learn more about the file's content or the program's purpose.



In spite of *reverse engineering* While a popular application, binary analysis is a much broader field than most people realize, as it encompasses several basic but crucial operations and many more advanced concepts such as *binary tools* or *dynamic stain analysis* This allows you to observe or even change the behavior of a program and monitor the data flow. Many tools are available, but two of the

most widely known and free dynamic binary tools are: **Intel batteries**

and **DynamoRIO** Open source.

Static analysis This includes checking executable binary code without actually running it. Static analysis tools can identify bugs or security vulnerabilities simply by reading the program.

- *Basic static analysis* This involves checking the executable file without considering the actual commands, so it's simple and potentially fast, but inefficient for large binary files and may miss important behaviors.
- *Advanced static analysis* This involves reverse engineering the internal components of binary code by loading the executable file into a program. *reverse engineer (disassembler)* to convert binary code into readable text to view program instructions, or *decompiler* To convert assembly-level syntax into higher-level abstractions for testing pseudocode, which is much more concise, often omitting certain details to make the code easier to understand. **IDA Pro** It is one of the most powerful and widely used decoders and reverse engineers.

Dynamic analysis involves observing the program as it executes in a realistic environment, or a nearly identical one if necessary,

using *simulations* system-wide or user-user mode with **QEMU** Dynamic analysis

tools integrate analysis code into the program to store information related to program execution as metadata.

- *Fundamental engine analysis techniques* This involves running the binary file and observing its behavior on the system, including system and library calls that are executed along with their arguments.
- *Advanced Dynamic Analysis* including usage *debugger* To examine the internal state of a running executable program, including the values of variables and the results of conditional branches. On Linux systems, **gdb** It is the most popular and extremely versatile tool, suitable for a wide variety of purposes.

```
linked_list.h
105     end->create(last_free_pos, args...);
106 }
107
108     template <typename Cond>
109     int remove_if (Cond&& condition) {
110         linked_list_node* current = begin;
111         linked_list_node* next;
112         linked_list_node* prev = nullptr;
113         int removed = 0;
114         bool block_empty;
115
116         while (current != nullptr) {
117             block_empty = true;
118             for (int i = 0; i < count; i++) {
119                 if (current->used(i)) {
120                     if (condition(current->to_const_reference(i))) {
```

```
multi-thre Thread 0x7ffff7b07 In: run test<1, 1, 1> L112 PC: 0x8004622
Breakpoint 2 at 0x800377a: linked_list.h:111. (12 locations)
(gdb) c
Continuing.

Breakpoint 2, linked_list<test_struct<1>, 1>::remove_if<void run test<1, 1, 1>(std::vector<
int, std::allocator<int> >&)::{lambda(test_struct<1> const&)#1}>(void run test<1, 1, 1>(std
::vector<int, std::allocator<int> >&)::{lambda(test_struct<1> const&)#1}&&) (
    this=0x7ffff7fedf00, condition=...) at linked_list.h:112
(gdb) █
```

It's important to note that these methods complement each other because static analysis is often only the first step, and further analysis is usually needed. Static analysis requires less time and can consider all execution paths in a program, while dynamic analysis is more resource-intensive and only considers a single execution path. However, dynamic analysis is generally more accurate because it works with real values. Often, combining the advantages of static and dynamic analysis in a hybrid method is the optimal solution.

When is binary analysis performed?

You can use binary analysis to examine and document the behavior of executable files for which you don't have the source code, which often happens with third-party libraries, drivers, and other system components that are only available as binary files.

Typically, software developers can create drivers for hardware devices based on their complete documentation, but many third-party manufacturers only provide a binary executable file, known as a driver *binary blob*. The precise operation and security quality of binary blobs are not fully understood, meaning they require users to trust third-party vendors with the security of their systems. Due to their nature, malware also requires *malware analysis*. Binary analysis is comprehensive

and reverse engineering. However, it can also be used to find problems, including security vulnerabilities that only appear at the binary level, and for advanced debugging, even when you have the source code.

Third-party security labs often use binary analysis to assess the security of applications, system software code, or embedded software on connected devices. The automotive and medical industries have stringent cybersecurity requirements, and many consumer electronics devices also undergo penetration testing performed by third parties or internally, where the testing methodology often attempts to "simulate" a real-world attacker. These assessments often follow a black-box or gray-box approach, implying that the source code is not available to security researchers. In such cases, binary analysis is the primary tool for detecting bugs, vulnerabilities, or non-compliance with cybersecurity requirements for those projects.

What are some basic binary analysis tools?

You can accomplish many seemingly difficult tasks by properly combining a few simple tools that are commonly pre-installed on most Linux systems. Below is a summary of some basic tools you can use to begin your binary analysis journey on Linux:

- For example, management utility **file** These tools are created to identify file types by searching for characteristic patterns within the file, such as special identification codes.
- **Binwalk** It's a similar tool created to search for embedded files and executable code within a given binary file. It's compatible with file signatures recognized by the file utility but also comes with a custom signature file for items commonly included in firmware images.
- You can check the details of the header section of a binary file using **lshreadelf** and use the program **ldd** to find out which shared objects that the binary file depends on and which library versions that the binary file expects.
- You can use a program that displays hex codes called **xxd** To display the bytes of a file in hexadecimal representation.
- You can use **nm** To list both static and dynamic symbols in a given binary file, object file, or shared object and decode the symbol names.
- **Use the strings tool.** Searching for strings within a binary file can be a simple way to learn about a program's functionality. Legitimate programs almost always include many strings, such as messages, URLs, or file locations. Encapsulated or hidden malware often contains very few strings.

- The specific functions used by an executable file provide clues about the program's functionality; therefore, examining the system and library calls executed by a binary file is possible. **strace** and **ltrace** can often give you a good overview of what a program is doing.

What are the challenges of analyzing binary code?

There are several reasons why analyzing binary code is more difficult than analyzing source code. For example, binary files often appear as large blocks, making it difficult to recover the high-level code structure. They contain a mixture of code and data that is easily misinterpreted. They often have symbols removed, making code comprehension significantly more challenging. The data types of variables are never explicitly stated, making the purpose and structure of the data difficult to discern. Furthermore, any small change to the code or data can easily corrupt the binary code. While it is possible to translate binary code into a human-readable format to some extent, we cannot fully recover the program's source code.